UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

Taller 1B en GitHub Codespaces — Kafka y Spark Streaming: los viajes en tiempo real

Docente: Heyder Medrano

Entorno: WEB: tu codespace de GitHub (4 núcleos, 16 GB), preparado con la «Preparación Previa — Taller 1 en GitHub Codespaces». Carpeta de trabajo: /workspaces/tpd-codespace/tpd/kafka.

Datos: Los eventos que publica la app de viajes y viajes reales de la TLC (junio de 2025) que el simulador publica como eventos. Tema de Kafka: «viajes».

Objetivo técnico: Construir el flujo en tiempo real del caso: la app publica cada paso del viaje en Kafka, un consumidor lo lee y se miden throughput y latencia, se comprueba que un consumidor caído no pierde eventos, y Spark Streaming calcula la espera por zona y los ingresos en ventanas de tiempo.

Términos: si un término no te resulta claro, búscalo en Glosario_Tecnico_Tecnologias_Procesamiento_Datos.docx (carpeta Sesion_1_Vie_30_Octubre/Estudiantes).

El problema

Cuando una pasajera pide un carro, la empresa tiene segundos para decidir qué conductor asignar y si sube el precio porque en esa zona hay más pasajeros que carros. El proceso por lotes del Taller 1A responde al día siguiente: llega tarde para esa decisión. Se necesita que cada evento del viaje llegue a todos los sistemas que lo usan sin perderse aunque uno se caiga, y medir la espera por zona mientras ocurre.

Parte	Qué se resuelve
A	Mover los eventos del viaje por Kafka, medir throughput y latencia, comprobar que un consumidor caído no pierde eventos y verificar la precisión del flujo.
B	Calcular en tiempo real la espera por zona y los ingresos por empresa con Spark Streaming.
Final	La solución: qué se logró.

Competencias que evidencia este taller

Competencia · resultado de aprendizaje	Dónde se evidencia	Evidencia que registras en la bitácora
C3 · RA3 — desarrollar un flujo de datos en tiempo real con Apache Kafka, garantizando su **funcionalidad**	Parte A, pasos 1 a 4; Parte B, paso 7	El viaje completo por Kafka, throughput y latencia, el offset en que retomó el consumidor, la espera por zona en ventanas
C3 · RA3 — … y su **precisión**	Parte A, paso 5	El resultado del verificador: eventos por tipo, duplicados y montos al centavo frente al archivo de la TLC

Qué corre y dónde

Pieza	Dónde corre	Qué hace
Kafka (broker) y ZooKeeper	Contenedores Docker del codespace	Guardan los eventos del tema «viajes», repartidos en 2 particiones.

Pieza	Dónde corre	Qué hace
app_viajes.py (Streamlit)	Python del codespace · se abre con la dirección que publica GitHub (puerto 8501)	Cliente, conductor y central: cada acción publica un evento en Kafka y las tres pantallas leen el estado desde Kafka.
consumidor_consola.py	Python del codespace	Lee el tema en tres modos: ver, medir y caída.
simulador.py	Python del codespace	Publica viajes reales de un día de la TLC como los mismos seis eventos de la app.
streaming_viajes.py	Contenedor spark	Spark Structured Streaming: ventanas de 15 minutos con watermark de 30.

Los seis eventos de un viaje, con la columna del archivo de la TLC que completa cada uno: solicitud (request_datetime) · asignado · llegada (on_scene_datetime) · recogida (pickup_datetime) · fin_viaje (dropoff_datetime, trip_miles) · pago (base_passenger_fare, tips, driver_pay). Todos llevan como clave el identificador del viaje.

- **Varias terminales:** este taller usa hasta tres a la vez (app, consumidor y simulador o streaming). Ábrelas con Ctrl + Mayús + ` (la tecla a la derecha de la Ñ) o con el + del panel de abajo; la lista de la derecha del panel permite pasar de una a otra con un clic. En cada terminal nueva, entra primero a la carpeta con `cd /workspaces/tpd-codespace/tpd/kafka`.

Parte A — Kafka y la app de viajes

En el caso: cuando alguien pide un viaje, la app del cliente, la del conductor y la central de operaciones necesitan enterarse de cada paso al instante. Kafka es el punto común: la app publica cada evento del viaje en el tema «viajes» y cada programa que lo necesita lo lee desde ahí. Se mide qué tan rápido funciona y se comprueba que no pierde eventos aunque un programa se caiga.

Prerrequisito: apagar Hadoop y encender Kafka

Para qué: liberar la memoria que usa el clúster de Hadoop del Taller 1A y encender el broker sabiendo que ya acepta conexiones.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $`

▶ **Qué hacer:** son 4 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 4

```
cd /workspaces/tpd-codespace/tpd/hadoop
```

Comando 2 de 4

```
docker compose stop
```

Comando 3 de 4

```
cd ../kafka
```

Comando 4 de 4

```
bash encender_kafka.sh
```

Salida esperada *(si Kafka quedó reiniciándose de una sesión anterior, el script lo recrea y espera un poco más)*

```
Encendiendo Kafka, ZooKeeper y Spark...
[+] Running 4/4
 ✓ Network kafka_default  Created
 ✓ Container zookeeper    Started
 ✓ Container spark        Started
 ✓ Container kafka        Started
  Kafka acepta conexiones (localhost:9092)                 OK (22 s)

KAFKA LISTO en 22 s · temas actuales:
```

✓ **Punto de control:** la salida termina en «KAFKA LISTO».

⚠ *Enciende Kafka siempre con bash encender_kafka.sh: así nunca abres la app, el consumidor o el simulador antes de que Kafka responda (eso produce «NoBrokersAvailable»).*

Paso 1. Abrir la app de viajes

Para qué: generar eventos reales de un viaje, con las mismas columnas que registra la TLC.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
streamlit run app_viajes.py --server.headless true
```

Salida esperada *(las direcciones IP serán otras: no las uses)*

```
  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.0.0.141:8501
  External URL: http://51.8.152.64:8501
```

Pulsa **Abrir en el navegador** en el aviso «La aplicación que se ejecuta en el puerto 8501 está disponible». Si el aviso ya se cerró, abre la dirección de tu codespace cambiando el final «.github.dev» por «-8501.app.github.dev».

⚠ *Deja esta terminal abierta: si presionas Ctrl+C, la app se detiene.*

- En la barra lateral, «Vista», deja «Vista integrada»: muestra cliente, conductor y central en la misma pantalla.
- Cliente: escribe tu nombre, elige empresa, zona de origen y de destino, y pulsa «Pedir viaje». El mapa marca el origen en verde y el destino en rojo.
- Conductor: antes de pedir el viaje, marca «Trabajo sin pareja: usar un conductor automático». Si trabajas con un compañero, él puede hacer de conductor en la misma pantalla: en la columna Conductor, con la misma empresa, pulsa «Aceptar» cuando aparezca la solicitud.
- Cuando el viaje termine, elige la propina y pulsa «Pagar (simulado)». La app nunca pide datos de tarjeta.



Figura 1. Viaje pagado desde la app publicada por GitHub: el cliente en «pagado» (1); la central recibió 6 eventos y cuenta 1 viaje pagado (2); tarifa US$ 21, pago US$ 16 y 23,0 % para la plataforma (3).

Lo que debes ver *(viaje de Midtown Center a LaGuardia Airport; tus montos dependen de las zonas)*

```
Cliente: Estado pagado · 12 de 12 columnas completas · ¡Pago recibido! Total US$ 22.16
Central: Eventos recibidos de Kafka 6 · Viajes solicitados 1 · Pagados 1 · Mensajes descartados 0
Tarifa base cobrada US$ 21 · Pago a conductores US$ 16 · Parte de la plataforma 23.0 %
```

Cómo leer el resultado: cliente y conductor nunca se hablan directamente: cada paso es un evento en Kafka y las tres pantallas se actualizan leyéndolo. Un viaje produce seis eventos y, al final, una fila completa como las del archivo de la TLC que procesaste en el Taller 1A.

Paso 2. Leer el tema desde la consola

Para qué: ver los eventos tal como los guarda Kafka: partición, offset y contenido.

Abre una terminal 2 (Ctrl + Mayús + `), entra a la carpeta y ejecuta el consumidor:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
cd /workspaces/tpd-codespace/tpd/kafka
```

Comando 2 de 2

```
python consumidor_consola.py
```

Salida esperada *(fragmento; tus identificadores, horas y la partición (0 o 1) serán otros)*

```
[1] partición 0 · offset 0 · viaje 8612040a · evento «solicitud»
     request_datetime=02:02:46 | on_scene_datetime=– | pickup_datetime=– | … (1/8 columnas)
[2] partición 0 · offset 1 · viaje 8612040a · evento «asignado»
[3] partición 0 · offset 2 · viaje 8612040a · evento «llegada»
[4] partición 0 · offset 3 · viaje 8612040a · evento «recogida»
[5] partición 0 · offset 4 · viaje 8612040a · evento «fin_viaje»
[6] partición 0 · offset 5 · viaje 8612040a · evento «pago»
     … | trip_miles=7.3 | base_passenger_fare=20.52 | tips=0.0 | driver_pay=15.8   (8/8 columnas)
```

Cómo leer el resultado: los seis eventos del viaje quedaron en la misma partición y en orden, porque todos llevan la misma clave: Kafka envía cada clave siempre a la misma partición y solo garantiza el orden dentro de una partición. El offset es la posición del evento en su partición.

Pide otro viaje en la app: aparece en la consola en cuanto lo pides. Luego detén el consumidor con Ctrl+C.

Para ver cómo quedó el tema: `kafka-topics --describe` muestra sus particiones, su líder y sus réplicas.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
docker compose exec kafka kafka-topics --bootstrap-server localhost:9092 --describe \
    --topic viajes
```

Salida esperada *(el TopicId cambia)*

```
Topic: viajes  TopicId: 6zDcoA_UQV2xYUAtlo33bQ       PartitionCount: 2      ReplicationFactor: 1
       Configs:
       Topic: viajes  Partition: 0   Leader: 1      Replicas: 1    Isr: 1
       Topic: viajes  Partition: 1   Leader: 1      Replicas: 1    Isr: 1
```

Cómo leer el resultado: 2 particiones con un solo broker (el 1), que es líder de ambas; ReplicationFactor 1 = una sola copia de cada evento. En producción se usan al menos 3 brokers y réplica 3.

Paso 3. Medir throughput y latencia con viajes reales

Para qué: cuantificar cuántos eventos por segundo atiende el flujo y cuánto tarda cada uno desde que se publica hasta que se lee (RA3).

Primero, en la terminal 2, el consumidor en modo medir: solo cuenta eventos nuevos, así que debe estar abierto antes del simulador. Termina solo 10 segundos después del último evento.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
python consumidor_consola.py medir
```

Luego, en una terminal 3 (Ctrl + Mayús + ` y cd a la carpeta), el simulador: toma 500 viajes reales del 2 de junio y publica sus 3.000 eventos, 100 por segundo.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
cd /workspaces/tpd-codespace/tpd/kafka
```

Comando 2 de 2

```
python simulador.py ../datos/fhvhv_tripdata_2025-06.parquet --dia 2025-06-02 \
    --viajes 500 --eventos-por-segundo 100
```

Salida esperada *(terminal 3)*

```
500 eventos enviados · hora simulada 00:00:41
…
3,000 eventos enviados · hora simulada 01:41:20
Listo: 500 viajes reales = 3,000 eventos en 31 s
```

Salida esperada *(terminal 2)*

```
Modo «medir» · grupo (ninguno) · esperando eventos del tema «viajes»... (Ctrl+C para salir)

500 eventos en 5.1 s · 97 eventos/s · latencia de los últimos 500: mediana 3 ms, máxima 30 ms
…
3,000 eventos en 30.8 s · 97 eventos/s · latencia de los últimos 500: mediana 2 ms, máxima 6 ms
Total leído en esta ejecución: 3,000 eventos
Throughput promedio: 97 eventos/s · latencia mediana 2 ms · percentil 95: 4 ms
```

Cómo leer el resultado: el consumidor lee al mismo ritmo que publica el simulador (97 de los 100 eventos por segundo pedidos) con una latencia mediana de 2 milisegundos: el límite lo pone el productor, no Kafka. La latencia se mide con la marca de tiempo que Kafka guarda en cada evento; productor y consumidor corren en el mismo codespace, así que el reloj es el mismo.

Paso 4. Tumbar un consumidor y comprobar que no se pierde nada

Para qué: comprobar que Kafka recuerda hasta dónde leyó cada grupo de consumidores (offset confirmado): si la central se cae, al volver retoma donde quedó.

En la terminal 2, el modo caída lee despacio (un evento cada 50 ms) dentro del grupo «central-caida». Ejecútalo, espera a ver al menos «200 eventos procesados» y presiona Ctrl+C:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
python consumidor_consola.py caida
```

Salida esperada *(la cantidad depende de cuándo presiones Ctrl+C)*

```
Modo «caida» · grupo central-caida · esperando eventos del tema «viajes»... (Ctrl+C para salir)

Primer evento de esta ejecución: partición 0, offset 0, viaje 8612040a, evento «solicitud»
100 eventos procesados · último: partición 0, offset 99
200 eventos procesados · último: partición 0, offset 199
^C
Detenido con Ctrl+C.
Total leído en esta ejecución: 210 eventos
```

Vuelve a ejecutar el mismo comando:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
python consumidor_consola.py caida
```

Salida esperada *(primeras líneas)*

```
Primer evento de esta ejecución: partición 0, offset 210, viaje tlc20250602_000198, evento «asignado»
^C
Detenido con Ctrl+C.
Total leído en esta ejecución: 47 eventos
```

✓ **Punto de control:** el primer offset de la segunda ejecución es exactamente el total leído en la primera (210 en la prueba): no se perdió ni se repitió ningún evento.

Detenlo con Ctrl+C. Para ver lo que Kafka tiene registrado del grupo: `kafka-consumer-groups --describe`.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
docker compose exec kafka kafka-consumer-groups --bootstrap-server localhost:9092 \
    --describe --group central-caida
```

Salida esperada *(columnas CONSUMER-ID, HOST y CLIENT-ID omitidas; tus cifras serán otras)*

```
Consumer group 'central-caida' has no active members.

GROUP          TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
central-caida  viajes  0          257             1464            1207
central-caida  viajes  1          0               1542            1542
```

Cómo leer el resultado: CURRENT-OFFSET es hasta dónde confirmó el grupo; LOG-END-OFFSET, cuántos eventos tiene la partición; LAG, cuántos le faltan por leer. En la partición 0 el grupo confirmó 257 = 210 + 47 (las dos ejecuciones). La partición 1 sigue en 0 porque el consumidor lento todavía no ha llegado a ella: no perdió nada, le faltan por leer. Entre las dos particiones hay 1.464 + 1.542 = 3.006 eventos: los 3.000 del simulador y los 6 de tu viaje en la app. Un lag que crece sin parar es la señal de que el consumidor no da abasto: los datos que ve la central (o un tablero) se están quedando atrás de la realidad.

Paso 5. Verificar la precisión del flujo

Para qué: demostrar que el flujo no solo funciona sino que es exacto (RA3): lo que llegó por Kafka debe coincidir con el archivo de origen.

`verificar_flujo.py` lee el tema desde el principio (sin grupo) y lo compara con los mismos viajes leídos del Parquet. Usa un día que no hayas publicado antes; aquí, el 5 de junio:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
python simulador.py ../datos/fhvhv_tripdata_2025-06.parquet --dia 2025-06-05 \
    --viajes 1000 --eventos-por-segundo 2000
```

Comando 2 de 2

```
python verificar_flujo.py ../datos/fhvhv_tripdata_2025-06.parquet --dia 2025-06-05 \
    --viajes 1000
```

Salida esperada

```
Listo: 1,000 viajes reales = 6,000 eventos en 6 s
Viajes del 2025-06-05 publicados por el simulador: 1,000 (origen: 1,000)
Eventos por tipo en Kafka: solicitud 1,000 · asignado 1,000 · llegada 1,000 · recogida 1,000 · fin_viaje
1,000 · pago 1,000
Eventos duplicados: 0

Empresa  Viajes (origen / Kafka)  Tarifa base US$ (origen / Kafka)  Pago conductor US$ (origen / Kafka)
Lyft     294 / 294                7,954.15 / 7,954.15              6,138.63 / 6,138.63
Uber     706 / 706                20,752.88 / 20,752.88            16,692.65 / 16,692.65

RESULTADO: PRECISO — ningún evento perdido ni duplicado y montos idénticos al centavo.
```

Cómo leer el resultado: cada viaje produjo sus seis eventos, ninguno llegó dos veces y los montos que recibió el flujo son idénticos a los del archivo de origen. Si el simulador publica el mismo día dos veces, el verificador lo detecta y hay que repetir con otro día.

✓ **Punto de control:** tu resultado dice «PRECISO».

Si algo falla en la Parte A

Lo que ves	Causa	Qué hacer
`kafka.errors.NoBrokersAvailable: NoBrokersAvailable`	Kafka no está encendido o quedó reiniciándose.	bash encender_kafka.sh y espera «KAFKA LISTO».
La app no abre con localhost:8501	Esa dirección es del codespace, no de tu navegador.	Usa «Abrir en el navegador» o la dirección …-8501.app.github.dev.
`FileNotFoundError: … fhvhv_tripdata_2025-06.parquet`	El simulador se ejecutó fuera de tpd/kafka.	cd /workspaces/tpd-codespace/tpd/kafka y repite.
El consumidor en modo medir no termina	Se abrió después de que el simulador terminó: espera eventos nuevos.	Ctrl+C; ábrelo primero y luego ejecuta el simulador.
La app no cambia al pedir el viaje	Nadie ha aceptado el viaje como conductor.	Acéptalo en la vista Conductor, o marca «Trabajo sin pareja» antes de pedir otro.

Parte B — Spark Streaming conectado a Kafka

En el caso: con los eventos fluyendo por Kafka, la gerencia quiere saber, mientras ocurre, en qué zonas están esperando más los pasajeros y cuánto ingresa cada empresa. Spark Streaming lee el tema «viajes» y cada 10 segundos actualiza esas cifras por ventanas de 15 minutos: es la señal que usaría una tarifa dinámica.

Paso 6. Revisar el job de streaming, parte por parte

Abre `tpd/kafka/streaming_viajes.py` en el explorador:

- `spark.readStream.format("kafka")…option("subscribe", "viajes")`: lee el tema como una tabla que no deja de crecer. Dentro de Docker, Kafka se llama kafka:29092 (desde el codespace es localhost:9092).
- `option("startingOffsets", "latest")`: procesa solo lo que llegue desde que arranca, como un flujo en tiempo real.
- `F.from_json(F.col("value").cast("string"), esquema)`: convierte cada mensaje JSON en columnas.
- `.withWatermark("event_time", "30 minutes")`: acepta eventos que lleguen hasta 30 minutos tarde; las ventanas más viejas se cierran y Spark libera su memoria.
- `groupBy(F.window("event_time", "15 minutes"), "PULocationID")`: ventanas de 15 minutos por zona de recogida.
- `.where(F.col("espera_min") >= 0)`: descarta las esperas negativas, el mismo error de datos que el MapReduce del Taller 1A contó como «Esperas negativas».
- `trigger(processingTime="10 seconds")`: cada 10 segundos procesa lo nuevo (un lote) y actualiza las dos tablas.

Paso 7. Ejecutar el streaming y alimentarlo

Para qué: calcular en tiempo real la espera por zona, la señal que usa una tarifa dinámica, y los ingresos por empresa.

⚠ El tema «viajes» debe existir antes de arrancar el streaming: si en esta sesión no has pedido ningún viaje ni corrido el simulador (por ejemplo, si reabriste el codespace), primero ejecuta el simulador una vez. Si no, el streaming falla con «UnknownTopicOrPartitionException».

En la terminal 2 (detén antes lo que tenga abierto con Ctrl+C):

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
docker compose exec spark /opt/spark/bin/spark-submit --conf \
    spark.jars.ivy=/tmp/.ivy2 --packages \
    org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.9 /opt/kafka-taller/streaming_viajes.py
```

Espera a ver «=== Lote 0 …»: sus tablas salen vacías porque el streaming solo procesa lo que llegue desde ahora. Con el streaming abierto, en la terminal 3 publica un día más de viajes:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
python simulador.py ../datos/fhvhv_tripdata_2025-06.parquet --dia 2025-06-04 \
    --viajes 300 --eventos-por-segundo 200
```

Salida esperada *(fragmento; el número del lote y las fechas cambian (las ventanas dicen el día de hoy))*

```
=== Lote 2 · INGRESOS por empresa (ventanas de 15 min) ===
|desde      |empresa|viajes_pagados|tarifa |pago_conductor|plataforma_pct|
|30/10 00:00|Lyft   |32            |353.32 |220.27        |37.7          |
|30/10 00:00|Uber   |67            |797.92 |500.74        |37.2          |
|30/10 00:15|Lyft   |42            |913.5  |714.62        |21.8          |
|30/10 00:15|Uber   |89            |1783.52|1482.47       |16.9          |
|30/10 00:30|Lyft   |12            |457.51 |355.43        |22.3          |
|30/10 00:30|Uber   |35            |1578.75|1319.59       |16.4          |
|30/10 00:45|Lyft   |2             |134.83 |102.62        |23.9          |
|30/10 00:45|Uber   |18            |831.36 |726.09        |12.7          |
|30/10 01:00|Uber   |1             |118.36 |90.75         |23.3          |
|30/10 01:15|Lyft   |1             |63.04  |70.75         |-12.2         |
|30/10 01:30|Lyft   |1             |118.79 |119.79        |-0.8          |
```

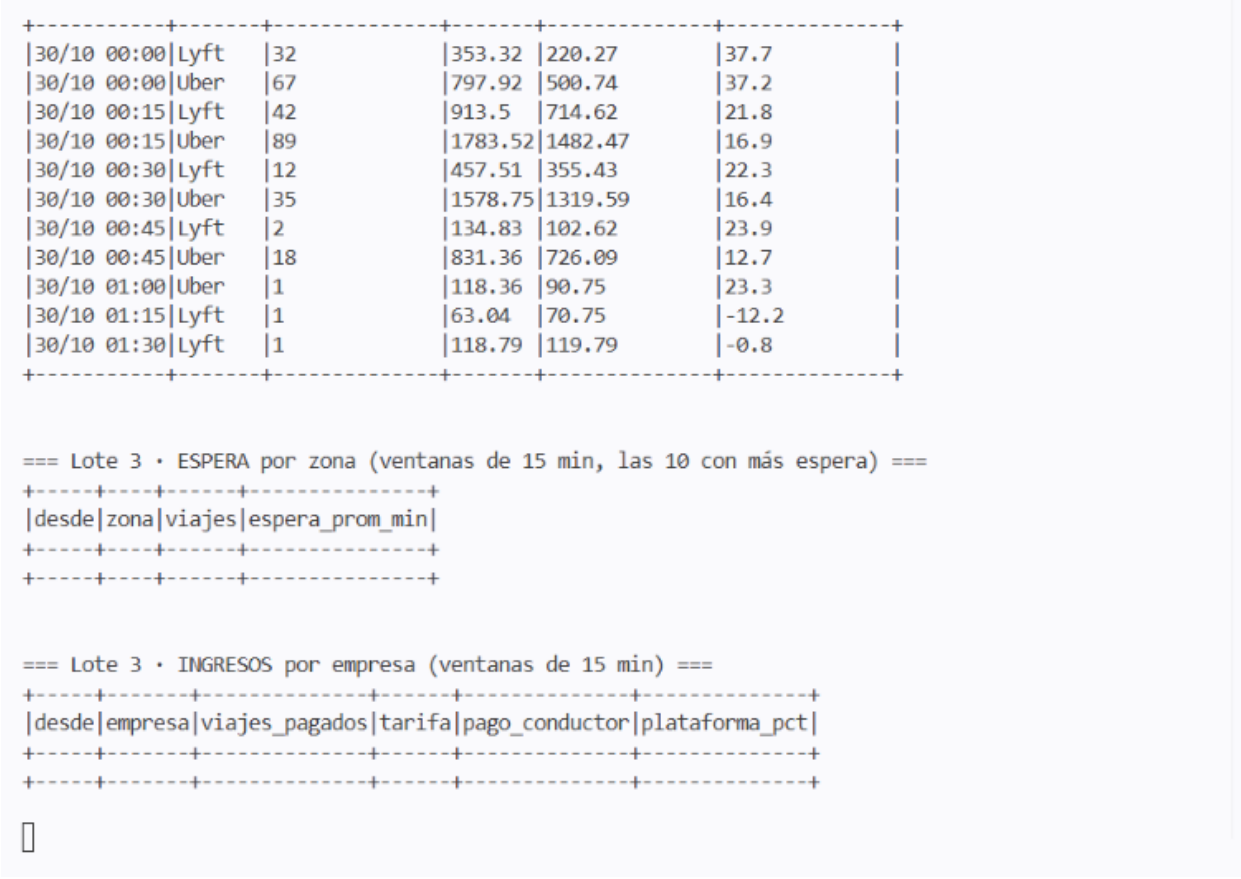


Figura 2. Spark Streaming en el codespace: ingresos por empresa en ventanas de 15 minutos; los viajes pagados suman 300.

Cómo leer el resultado: el simulador traslada los viajes a la fecha de hoy conservando su hora, por eso las ventanas dicen el día de hoy y empiezan a las 00:00. Cada lote muestra solo las ventanas que cambiaron; en la tabla de ingresos los viajes pagados suman 300 (32 + 67 + 42 + 89 + 12 + 35 + 2 + 18 + 1 + 1 + 1), los 300 del día simulado. La tabla de espera por zona sale arriba de la de ingresos en cada lote: sube en la terminal para verla. Un porcentaje negativo para la plataforma sale en ventanas de un solo viaje en que el pago al conductor superó la tarifa base; con muchos viajes el promedio se estabiliza.

Registra en la bitácora, con la última tabla de espera por zona: ¿qué zona tiene la mayor espera promedio en la ventana más reciente?, ¿con cuántos viajes?, y ¿qué decisión tomaría la central (subir el precio, mover conductores hacia esa zona) y con qué riesgo si la ventana tiene muy pocos viajes?

Para detener el streaming: Ctrl+C en su terminal.

Paso 8. Apagar

Para qué: liberar memoria y dejar Kafka listo para la próxima sesión.

1. Cierra la pestaña de la app y presiona Ctrl+C en las terminales de la app, del streaming y de los consumidores.
2. Apaga Kafka con down: borra sus contenedores (y los eventos del tema), así la próxima vez arranca limpio.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/kafka · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/kafka (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
docker compose down
```

3. Apaga el codespace: F1 → «Codespaces: Stop Current Codespace». Encendido sigue consumiendo horas de tu cuota.

La solución

Necesidad	Cómo quedó resuelta	Evidencia del taller (codespace)
Que cada evento del viaje llegue a todos los sistemas	La app publica cada paso en el tema «viajes»; la app, la central, el consumidor y Spark leen del mismo tema sin conocerse entre sí.	Un viaje = 6 eventos en la misma partición y en orden; la fila de la TLC se completa (12 de 12 columnas).
Que el flujo aguante el volumen y responda rápido	Kafka recibe y entrega los eventos en milisegundos.	98 eventos/s al ritmo pedido, con latencia mediana de 2 ms y percentil 95 de 3 ms.
Que una caída no pierda eventos	Cada grupo de consumidores confirma su offset; al volver, retoma donde quedó.	La segunda ejecución empezó exactamente en el offset 210, donde quedó la primera.
Que el flujo sea exacto	Se compara lo que llegó por Kafka con el archivo de origen.	1.000 viajes, 6.000 eventos, 0 duplicados y montos idénticos al centavo.
Saber dónde está subiendo la espera, mientras ocurre	Spark Streaming agrega por ventanas de 15 minutos, con watermark de 30.	Cada 10 segundos: espera por zona e ingresos por empresa.

Paso 9. Registrar (insumo para el Proyecto Final)

En tu bitácora técnica registra: throughput y latencia medidos y dónde estuvo el límite; el offset en que retomó el consumidor y qué lo garantiza; el resultado del verificador; qué zonas y horas tuvieron más espera y qué haría con eso una tarifa dinámica; y qué cambiaría para operar esto en producción (brokers, réplica, particiones, consumidores por grupo).